

多模态图文检索与 RAG 评测 Pipeline

项目学习讲义与面试深挖手册

项目路径: /home/students/liym/MultimodalRAG

2026 年 7 月 9 日

本文用途

这是一份面向打印学习、面试复盘和项目深挖的讲义。它把当前 Demo 拆成一个完整经历闭环：为什么要做、怎么做、每一步技术原理是什么、为什么这样选、代码里如何落地、如何评测、面试官会怎么追问。

目录

1	先定事实边界：这是什么项目，不是什么项目	4
1.1	项目定位	4
1.2	不能夸大的边界	4
1.3	服务器与代码位置	4
2	完整故事：从业务问题到技术闭环	4
2.1	业务背景	4
2.2	项目要解决的问题	5
2.3	闭环流程	5
3	数据层：为什么要把图文资料拆成多个字段	5
3.1	文档 Schema	5
3.2	每个字段的意义	6
3.3	选择理由	6
4	Query 路由：为什么纯文本、单图、多图、图文组合要分开	6
4.1	Query Schema	6
4.2	路由规则	7
4.3	多图为什么不能简单拼接	7
5	BM25：关键词召回为什么仍然重要	7
5.1	原理	7
5.2	代码对应	8
5.3	为什么选择 BM25	8

6 Dense Embedding：语义召回解决什么问题	8
6.1 原理	8
6.2 代码对应	8
6.3 当前 Demo 怎么实现	8
7 Image Embedding、OCR 与 Caption：图文检索的关键桥梁	9
7.1 为什么图片不能只存路径	9
7.2 本项目中的做法	9
7.3 为什么 OCR/Caption 要进入文本索引	9
8 多路融合：为什么需要 RRF 或加权归一化	9
8.1 问题：不同召回器分数不可直接比较	9
8.2 方案一：Score Normalization + Weighted Fusion	10
8.3 方案二：RRF	10
8.4 为什么本项目默认 RRF	10
9 Reranker：为什么要二阶段排序	10
9.1 两阶段检索思想	10
9.2 代码对应	11
9.3 为什么当前接口是合理的	11
10 评测体系：如何证明检索真的变好	11
10.1 为什么必须评测	11
10.2 Hit@K	11
10.3 MRR	11
10.4 nDCG@10	12
10.5 代码对应	12
10.6 当前示例评测结果	12
11 RAG 层指标：从检索结果到答案质量	12
12 Badcase 回溯：项目是否经得起深挖的关键	13
12.1 为什么 Badcase 比平均分更重要	13
12.2 Demo 中的 Badcase 表	13
12.3 建议的失败归因表	13
13 代码走读：从命令到函数调用	14
13.1 生成示例数据	14
13.2 构建索引	14
13.3 执行检索	14
13.4 执行评测	15

14 如何把轻量 Demo 升级成更真实的项目	15
14.1 第一步：替换 Embedding 模型	15
14.2 第二步：接入真实 OCR/Caption	16
14.3 第三步：接入向量库或 ES	16
14.4 第四步：增加 LLM Answer 与 RAG 评估	16
15 面试深挖问答	16
15.1 Q1: 你为什么不只用向量检索?	16
15.2 Q2: 多图检索怎么做?	16
15.3 Q3: RRF 和加权融合怎么选?	16
15.4 Q4: Reranker 为什么放在后面?	16
15.5 Q5: 你怎么评估检索效果?	17
15.6 Q6: 这个项目和 RAG 有什么关系?	17
15.7 Q7: 你这个 Demo 的不足是什么?	17
16 最终项目叙事	17
17 复习清单	17

1 先定事实边界：这是什么项目，不是什么项目

1.1 项目定位

本项目是一个**多模态图文检索与 RAG 评测 Demo**。它不是线上生产系统，而是为了补齐和复盘“多模态检索、RAG 评测、Badcase 归因”能力而实现的最小可运行闭环。

可以对外描述为：

我实现了一个多模态图文检索与 RAG 评测 Demo，将文本 BM25 召回、Dense Embedding 语义召回、Image Embedding 图像召回、OCR/Caption 文本索引、多路 RRF 融合、Reranker 精排、Hit@K/MRR/nDCG 评测和 Badcase 回溯串成完整闭环。它可以替换为服务器上的 BGE/Qwen 模型，用于模拟真实多模态 RAG 系统的检索评估流程。

1.2 不能夸大的边界

边界提醒：这个 Demo 可以用于学习和面试展示，但不要说成“已经服务百万级真实用户”“线上系统指标达到某某数值”“完整接入真实 ES/Milvus/Qwen 多模态生产链路”。目前已经真实实现的是：可运行检索链路、示例数据、评测指标、Badcase 输出、服务器部署。

1.3 服务器与代码位置

项目项	路径或说明
服务器路径	/home/students/liym/MultimodalRAG
本地路径	/Users/ououmeng/Documents/GetWork/MultimodalRAG
入口命令	python3 -m multimodal_rag_demo ...
Web UI	python3 app.py, 默认端口 7860
评测报告	reports/eval_report.json
BGE 路径	/home/students/liym/bge_m3
Qwen 路径	/home/students/liym/Qwen3.5_9B/Qwen3.5_9B

2 完整故事：从业务问题到技术闭环

2.1 业务背景

真实业务中，知识资料很少只有纯文本。一个企业内部知识库里常见的资料包括：

- 文档正文：制度、说明、论文摘要、项目记录、FAQ。
- 图片和截图：流程图、系统界面、PPT 截图、PDF 页面。
- 图片中的文字：OCR 后才能被文本检索命中。

- 图片语义：图里画的是流程、指标表、报错页面还是架构图。
- 标题和链接：很多时候标题、URL、文件名包含强线索。

如果只做关键词检索，用户换一种说法就可能找不到；如果只做向量检索，又可能召回“语义像但事实不对”的内容；如果图片只当作文件附件，不做 OCR、Caption 或图像向量，RAG 就无法真正理解图文混合资料。

2.2 项目要解决的问题

本项目把问题拆成四层：

1. **资料如何入库**：同一份资料要保留标题、正文、OCR、Caption、URL、图片引用等字段。
2. **Query 如何路由**：纯文本、单图、多图、图文组合不能走同一条检索路径。
3. **候选如何融合**：BM25、Dense、Image、标题链接各有分数，必须统一排序。
4. **效果如何评估**：不能只看“结果看起来不错”，要用 Hit@K、MRR、nDCG 和 Badcase 回溯。

2.3 闭环流程

业务资料

- > 字段化入库：title / body / ocr / caption / url / image_ref
- > 建索引：BM25 + text embedding + image embedding
- > Query 路由：text / single image / multi image / image+text
- > 多路召回：关键词召回 + 语义召回 + 图像召回 + 标题链接召回
- > 候选融合：RRF 或 weighted fusion
- > Reranker 精排：保留接口，可替换真实 BGE reranker
- > Top-K 输出
- > 指标评测：Hit@K / MRR / nDCG@10
- > Badcase 回溯：定位数据、OCR、路由、召回、融合、重排问题

学习检查：如果面试官问“你这个项目到底解决什么”，一句话回答：解决图文混合知识库中多路召回、排序融合和可量化评测的问题。

3 数据层：为什么要把图文资料拆成多个字段

3.1 文档 Schema

代码位置：multimodal_rag_demo/schema.py

```
@dataclass
class Document:
    doc_id: str
    title: str
    body_text: str = ""
    ocr_text: str = ""
    caption: str = ""
```

```
url: str = ""
image_ref: str = ""
source_type: str = "doc"
```

3.2 每个字段的意义

字段	作用	为什么需要
title	标题、文件名、页面名	标题通常是强语义线索，适合关键词和标题匹配。
body_text	正文内容	Dense 语义检索和 BM25 检索的主内容。
ocr_text	图片或截图中的文字	把视觉资料中的文字转成可检索文本。
caption	图片语义描述	弥合图像向量和文本语义之间的差距。
url	来源链接	用于引用、溯源、链接可用率评估。
image_ref	图片路径或标识	用于图像向量检索或多图 Query。
source_type	来源类型	便于按 workflow、paper、metrics 等类型过滤。

3.3 选择理由

单字段入库会把信息揉在一起，后续很难判断是标题命中、正文命中、OCR 命中还是图片语义命中。多字段设计让评测更可解释：一个错误结果到底是 OCR 噪声、标题误导，还是 embedding 召回不准，可以分别回溯。

4 Query 路由：为什么纯文本、单图、多图、图文组合要分开

4.1 Query Schema

代码位置：multimodal_rag_demo/schema.py

```
@dataclass
class Query:
    query_id: str
    text: str = ""
    image_refs: List[str] = field(default_factory=list)
    relevant_doc_ids: List[str] = field(default_factory=list)
    query_type: Optional[str] = None
```

4.2 路由规则

Query 类型	检索路径	设计理由
纯文本	BM25 + Dense + 标题链接	同时覆盖精确词和语义改写。
单图	Image Embedding + OCR/Caption 文本	图像本身和图中文字都可能是线索。
多图	多图向量平均或分路召回后融合	多张图可能表达同一主题, 也可能各有信息点。
图文组合	文本、图像、标题链接多路打分	用户文字约束和图片语义需要共同参与排序。

4.3 多图为什么不能简单拼接

多图 Query 的难点是：每张图的信息权重不一定相等。有时几张图都指向同一个主题，可以对 image embedding 做平均池化；有时几张图分别代表不同方面，比如“报错截图 + 架构图 + 指标表”，这时每张图单独召回，再用 RRF 融合更稳。

学习检查：面试回答模板：我先判断 Query 类型，再决定走 BM25、Dense、Image 或多路组合。多图场景不会直接拼图，而是单独编码后平均池化或分路召回融合。

5 BM25：关键词召回为什么仍然重要

5.1 原理

BM25 是经典稀疏检索方法。它关注三个问题：

- Query 词是否出现在文档里。
- 词在该文档中出现得是否足够多。
- 这个词在整个语料里是否足够稀有。

本项目中的简化公式：

$$\text{score}(D, Q) = \sum_{q_i \in Q} \text{idf}(q_i) \cdot \frac{f(q_i, D)(k_1 + 1)}{f(q_i, D) + k_1(1 - b + b \cdot |D|/\text{avgdl})}$$

其中：

- $f(q_i, D)$ ：词 q_i 在文档 D 中出现次数。
- $\text{idf}(q_i)$ ：词越稀有，权重越高。
- $|D|/\text{avgdl}$ ：对文档长度做归一化，避免长文档天然占便宜。

5.2 代码对应

代码位置: `retrieval.py` -> `BM25Index`

```
idf = math.log(1 + (self.n - df + 0.5) / (df + 0.5))
denom = freq + self.k1 * (1 - self.b + self.b * dl / max(self.avgdl, 1e-9))
score += idf * (freq * (self.k1 + 1)) / denom
```

5.3 为什么选择 BM25

BM25 对以下情况非常有效:

- 论文标题、会议名、模型名、文件编号等精确词。
- 报错文本, 如 `Illegal content`、`PDF download failed`。
- 专有名词, 如 `RRF`、`nDCG`、`NeurIPS`、`Agent` 工作流。

Dense embedding 会把语义相近内容拉近, 但对精确词可能不如 BM25 稳。因此检索系统里通常不会只用向量检索。

6 Dense Embedding: 语义召回解决什么问题

6.1 原理

Dense Embedding 将文本编码成一个稠密向量。语义相近的文本在向量空间中距离更近。检索时计算 Query 向量和文档向量的余弦相似度:

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

6.2 代码对应

代码位置: `models.py` 与 `retrieval.py`

```
q_vec = self.models.text_embedding.encode([text])[0]
return {
    doc.doc_id: cosine(q_vec, vec)
    for doc, vec in zip(self.docs, self.text_vectors)
}
```

6.3 当前 Demo 怎么实现

当前服务器没有安装 `sentence-transformers` 和 `transformers` 时, 会自动使用 hashing embedding 兜底。它不是语义模型, 但能保证流程形状完整: 编码、向量化、余弦相似度、多路融合、评测全部可运行。

如果安装依赖并配置模型路径, 代码会尝试加载:

```
BGE_EMBEDDING_MODEL=/home/students/liym/bge_m3
QWEN_MODEL=/home/students/liym/Qwen3.5_9B/Qwen3.5_9B
```

边界提醒：面试中应说：当前 Demo 已保留本地 BGE/Qwen 模型适配接口；如果依赖齐全，可替换 hashing embedding。不要说“当前已经完整跑通真实 Qwen 多模态推理”，除非后续确实安装并验证。

7 Image Embedding、OCR 与 Caption：图文检索的关键桥梁

7.1 为什么图片不能只存路径

如果只把图片作为附件保存，检索系统无法理解图片内容。图文资料至少有三种信息：

- 视觉内容：截图、架构图、表格、流程节点。
- 图片中文字：OCR 可以转为可检索文本。
- 图片语义描述：Caption 可以转为更自然的文本表达。

7.2 本项目中的做法

代码位置：Document.image_embedding_text() 和 SearchEngine.image_scores()

```
def image_embedding_text(self) -> str:
    return " ".join(part for part in [self.image_ref, self.title, self.caption, self.
        ocr_text] if part)
```

轻量 Demo 里没有真实图片模型，所以把 image_ref + title + caption + ocr_text 拼接后编码，模拟图像语义通道。真实系统中这里可以替换为 Qwen-VL、CLIP 或其他多模态 embedding 模型。

7.3 为什么 OCR/Caption 要进入文本索引

假设用户搜“格式转换失败”，但这几个字只出现在截图里。如果不做 OCR，BM25 和文本 embedding 都无法命中。Caption 则适合补充“这张图是什么”的语义，比如“ workflow 节点图”“评测指标表”“报错截图”。

8 多路融合：为什么需要 RRF 或加权归一化

8.1 问题：不同召回器分数不可直接比较

BM25、余弦相似度、图像相似度、标题链接重合度的分数尺度完全不同：

- BM25 可能是 0 到 10 甚至更高。
- 余弦相似度通常在 -1 到 1。
- 标题链接重合度可能是 0 到 1。

如果直接相加，分数大的通道会压制其他通道。

8.2 方案一：Score Normalization + Weighted Fusion

先把每一路分数归一化到可比较区间，再加权：

$$\text{score}(d) = w_1 s_{\text{bm25}}(d) + w_2 s_{\text{dense}}(d) + w_3 s_{\text{image}}(d) + w_4 s_{\text{link}}(d)$$

当前代码中的权重：

```
weighted_fuse([(bm25, 0.35), (dense, 0.3), (image, 0.25), (link_title, 0.1)])
```

适用场景：各路分数较稳定、可以校准权重。

8.3 方案二：RRF

RRF 全称 Reciprocal Rank Fusion。它不关心原始分数，只关心候选在每一路召回列表中的排名：

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

代码位置：retrieval.py -> rrf_fuse

```
for rank, (doc_id, _score) in enumerate(ranked, start=1):
    fused[doc_id] += 1.0 / (k + rank)
```

适用场景：不同召回器分数分布差异很大，直接归一化不稳定。

8.4 为什么本项目默认 RRF

因为 Demo 同时有 BM25、Dense、Image、标题链接四路分数，它们尺度差异明显。RRF 用排名融合，能减少分数尺度不一致带来的风险，适合教学和系统初版。

学习检查：面试时可以说：如果各路分数可校准，我用加权融合；如果不同召回器尺度差异大，我优先用 RRF，因为它按排名融合，更稳健。

9 Reranker：为什么要二阶段排序

9.1 两阶段检索思想

真实系统通常不会对全库做复杂模型重排，因为成本太高。常见做法是：

1. 第一阶段：BM25 / 向量检索快速召回 TopK 或 TopN。
2. 第二阶段：Reranker 对候选逐对精排。

第一阶段追求召回率和速度，第二阶段追求排序质量。

9.2 代码对应

代码位置: `SearchEngine.rerank`

```
if self.models.reranker is None or not query.text:
    return results
```

当前 Demo 保留接口。如果配置 BGE reranker 并安装依赖, 就可以将 Query-Document pair 送入 CrossEncoder 打分。

9.3 为什么当前接口是合理的

项目的重点不是“立刻上线最强 reranker”, 而是把系统边界预留好。这样你在面试中可以讲清楚:

- 初召回阶段如何保证候选覆盖。
- 重排阶段为什么放在 TopK 后。
- 如何从 lightweight Demo 平滑替换到 BGE reranker。

10 评测体系: 如何证明检索真的变好

10.1 为什么必须评测

RAG 项目很容易陷入“看几个例子感觉不错”。但是面试官会追问:

- Top5 里有没有正确资料?
- 第一个正确结果排在第几?
- 多路融合有没有比单路更好?
- 哪些 Query 失败, 失败原因是什么?

所以需要固定 Query 集和指标。

10.2 Hit@K

Hit@K 判断 TopK 中是否出现相关文档:

$$\text{Hit@K} = \begin{cases} 1, & \text{TopK 中有相关文档} \\ 0, & \text{TopK 中无相关文档} \end{cases}$$

适合回答: 系统有没有把正确资料召回来。

10.3 MRR

MRR 关注第一个相关结果的位置:

$$\text{MRR} = \frac{1}{\text{rank}_{first}}$$

如果正确结果排第一, MRR=1; 排第二, MRR=0.5。它适合衡量“用户一眼能不能看到正确结果”。

10.4 nDCG@10

DCG 考虑相关性和排序位置：

$$\text{DCG@K} = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}$$

nDCG 是把 DCG 除以理想排序下的 IDCG，范围通常在 0 到 1。它适合多等级相关性标注。当前 Demo 用 0/1 相关性，也能演示排序质量。

10.5 代码对应

代码位置：evaluate.py

```
def hit_at(k: int) -> float:
    return 1.0 if any(doc_id in relevant for doc_id in ranked_ids[:k]) else 0.0

for idx, doc_id in enumerate(ranked_ids, start=1):
    if doc_id in relevant:
        mrr = 1.0 / idx
        break
```

10.6 当前示例评测结果

示例数据共 5 条 Query，当前评测结果：

指标	当前值	解释
Hit@1	0.8	5 条 Query 中 4 条 Top1 命中。
Hit@5	1.0	所有 Query 的 Top5 都包含相关文档。
Hit@10	1.0	所有 Query 的 Top10 都包含相关文档。
MRR	0.9	第一个相关结果整体比较靠前。
nDCG@10	0.926	排序质量较好，但仍有优化空间。

边界提醒：这些数值是 Demo 示例数据的结果，不应写成真实历史业务结果。简历里可以写“实现评测指标与示例评测闭环”，不要写“真实项目 Top5 命中率达到 100%”。

11 RAG 层指标：从检索结果到答案质量

当前 Demo 已输出 RAG 指标占位，后续接入 LLM Judge 或人工标注后可扩展：

指标	关注问题	例子
Context Precision	召回上下文是否大多相关	TopK 里是否混入大量无关文档。
Context Recall	关键证据是否召回完整	问格式转换失败，是否召回失败日志。
Faithfulness	答案是否忠实上下文	答案是否编造了文档里没有的结论。
Answer Relevancy	答案是否回答问题	用户问检索链路，答案是否偏到模型训练。

这部分用于连接“检索评测”和“RAG 生成评测”。面试时可以说：先评检索，因为没有好上下文就没有好答案；再评生成，确保答案忠实于上下文。

12 Badcase 回溯：项目是否经得起深挖的关键

12.1 为什么 Badcase 比平均分更重要

平均指标只能说明整体趋势，Badcase 才能指导迭代。一个检索系统失败，可能不是模型问题，而是：

- 文档没有入库。
- OCR 识别错了。
- Query 路由错了。
- BM25 字段没覆盖。
- Embedding 对领域词不敏感。
- RRF 融合让错误通道权重过高。
- Reranker 把语义相似但事实错误的结果排前。

12.2 Demo 中的 Badcase 表

代码位置：evaluate.py

```
badcases.append({
    "query_id": query.query_id,
    "query_type": query.infer_type(),
    "failure_stage": "recall",
    "failure_reason": "Top10_无人工标注相关文档",
    "next_action": "检查_BM25_字段、embedding_表达、OCR/caption_质量或融合权重",
})
```

12.3 建议的失败归因表

失败阶段	现象	处理动作
数据入库	文档缺失、字段为空	检查 JSONL、字段映射、清洗规则。
OCR/Caption	图片文字错误或语义空泛	替换 OCR 模型，增加人工抽检。
Query 路由	多图 Query 走成纯文本	优化 query_type 判断。
BM25 召回	精确词没命中	增加字段、同义词、分词策略。
Dense 召回	语义相近但事实错	换 BGE/Qwen embedding，增加领域数据。
融合排序	某一路召回压制其他结果	调整权重或改用 RRF。
Reranker	前排结果事实错误	换 reranker，增加负样本。
RAG 生成	答案脱离上下文	强制引用、拒答规则、Faithfulness 评估。

13 代码走读：从命令到函数调用

13.1 生成示例数据

```
python3 -m multimodal_rag_demo sample-data \
  --docs data/raw/sample_docs.jsonl \
  --queries data/raw/sample_queries.jsonl
```

对应文件：`sample_data.py`。它生成 5 条文档和 5 条 Query，用于演示 Agent workflow、论文检索、多模态检索和评测指标。

13.2 构建索引

```
python3 -m multimodal_rag_demo build \
  --data data/raw/sample_docs.jsonl \
  --index data/index/demo_index.json
```

调用链路：

```
cli.py -> cmd_build
-> load_documents
-> SearchEngine.build
-> text_embedding.encode
-> image_embedding.encode
-> engine.save
```

13.3 执行检索

```
python3 -m multimodal_rag_demo search \  
  --index data/index/demo_index.json \  
  --query "BM25_dense_embedding_RRF_reranker" \  
  --image multimodal_pipeline.png \  
  --top-k 3
```

调用链路:

```
cli.py -> cmd_search  
  -> SearchEngine.load  
  -> SearchEngine.search  
    -> bm25.score  
    -> dense_scores  
    -> image_scores  
    -> link_title_scores  
    -> rrf_fuse / weighted_fuse  
    -> rerank
```

13.4 执行评测

```
python3 -m multimodal_rag_demo eval \  
  --index data/index/demo_index.json \  
  --queries data/raw/sample_queries.jsonl \  
  --out reports/eval_report.json
```

调用链路:

```
cli.py -> cmd_eval  
  -> load_queries  
  -> evaluate  
    -> engine.search  
    -> evaluate_query  
      -> Hit@K / MRR / nDCG@10  
    -> badcases  
  -> write_json
```

14 如何把轻量 Demo 升级成更真实的项目

14.1 第一步：替换 Embedding 模型

当前 fallback 是 hashing embedding。升级时优先替换为本地 BGE:

- 安装 sentence-transformers。
- 配置 BGE_EMBEDDING_MODEL=/home/students/liym/bge_m3。
- 重新 build index。

14.2 第二步：接入真实 OCR/Caption

可以新增数据预处理脚本：

- 图片 -> OCR 文本。
- 图片 -> Caption 描述。
- PDF/PPT 页面 -> 图片 + OCR + 文本。

14.3 第三步：接入向量库或 ES

当前 Demo 把向量和文档存成 JSON，适合学习。真实规模下可替换为：

- Elasticsearch：BM25、字段过滤、RRF hybrid search。
- Milvus / FAISS：向量索引。
- SQLite / DuckDB：小型离线实验元数据。

14.4 第四步：增加 LLM Answer 与 RAG 评估

当前评的是检索 TopK。后续可以加：

- 根据 TopK context 生成答案。
- 强制答案引用 doc_id 或 URL。
- 用人工标注或 LLM Judge 评 Faithfulness、Answer Relevancy。

15 面试深挖问答

15.1 Q1：你为什么不只用向量检索？

答：向量检索擅长语义召回，但对专有名词、论文标题、报错码、文件编号不一定稳定。BM25 对精确词更敏感，所以我用 BM25 + Dense 双路召回。两者失败模式不同，组合后更稳。

15.2 Q2：多图检索怎么做？

答：我先对每张图独立编码。如果多图表达同一主题，可以对 image embedding 做平均池化；如果多图表达不同信息点，则每张图分路召回，再用 RRF 融合。这样避免某一张图过度主导结果。

15.3 Q3：RRF 和加权融合怎么选？

答：如果各路分数能校准，比如 BM25、Dense、Image 都能映射到稳定区间，可以用加权融合。如果分数尺度差异大，我倾向 RRF，因为它按排名融合，不直接比较原始分数，更稳健。

15.4 Q4：Reranker 为什么放在后面？

答：Reranker 成本高，不能对全库跑。先用 BM25/向量快速召回 TopK 或 TopN，再用 reranker 精排，是效率和质量的折中。

15.5 Q5: 你怎么评估检索效果?

答: 我构造 Query 集, 每条标注相关 doc_id。用 Hit@K 看 TopK 有没有命中, 用 MRR 看第一个相关结果的位置, 用 nDCG@10 看排序质量。然后导出 Badcase, 看失败发生在数据、OCR、路由、召回、融合还是重排。

15.6 Q6: 这个项目和 RAG 有什么关系?

答: RAG 的第一步是检索。如果检索不到正确上下文, 后面生成再强也容易幻觉。本项目先把多模态资料召回和排序评估做扎实, 再预留 Context Precision、Context Recall、Faithfulness、Answer Relevancy 等 RAG 指标。

15.7 Q7: 你这个 Demo 的不足是什么?

答: 当前 Demo 是最小闭环, 尚未完整接入真实 BGE/Qwen 推理、真实 OCR/Caption、ES/Milvus 和 LLM Answer 生成。它的价值在于把系统骨架、评测指标和 Badcase 回溯跑通, 后续可以逐步替换每个模块。

16 最终项目叙事

可以用下面这段作为完整项目故事:

这个项目的背景是, 图文混合知识库中资料来源复杂, 既有正文文本, 也有截图、流程图、OCR 文字、Caption、标题和链接。单一路径检索很难同时覆盖精确关键词、语义相似内容和图像信息, 因此我实现了一个多模态图文检索与 RAG 评测 Demo。项目先将文档结构化 title、body、ocr、caption、url、image_ref 等字段, 再分别构建 BM25 文本召回、Dense Embedding 语义召回、Image Embedding 图像召回和标题链接召回。对于纯文本、单图、多图、图文组合 Query, 系统会走不同路由, 并通过 RRF 或加权归一化融合候选, 预留 reranker 精排接口。最后用 Hit@K、MRR、nDCG@10 评估 Top-K 质量, 并导出 Badcase, 定位失败发生在数据入库、OCR、路由、召回、融合还是重排阶段。这个项目让我完整理解了 RAG 系统中“检索质量决定生成上限”的工程逻辑。

17 复习清单

打印后可按下面清单自测:

1. 能否一句话解释项目解决什么问题?
2. 能否画出数据入库到 TopK 输出的完整链路?
3. 能否解释 BM25 公式中 IDF 和文档长度归一化的意义?
4. 能否解释 Dense Embedding 和余弦相似度?
5. 能否说明 OCR/Caption 为什么必须进入文本索引?
6. 能否讲清 RRF 为什么比直接加权更稳?

7. 能否解释 reranker 为什么放在二阶段?
8. 能否手算一个简单 Hit@K 和 MRR?
9. 能否说出 nDCG@10 衡量什么?
10. 能否列出 5 类 Badcase 及排查动作?
11. 能否诚实说明当前 Demo 和真实生产系统的差距?
12. 能否把这个项目自然连接到 Agent/RAG 实习经历?